Proiectul 2

Programare Orientată pe Obiecte

Grupele 141 și 144

Obiectiv

Extinderea aplicației C++ realizate în Proiectul 1 (sau realizarea unei aplicații noi) prin integrarea conceptelor de **compunere**, **moștenire**, **funcții virtuale**, **clase abstracte** și **polimorfism**, conform materiei studiate în Laboratoarele 4 și 5.

Proiectul 2 **continuă în mod direct Proiectul 1**: toate cerințele obligatorii ale Proiectului 1 (încapsulare, constructori, destructori, operatori supraîncărcați, meniu CRUD etc.) rămân valabile și trebuie să fie prezente în codul predat.

Cerințe obligatorii

1. Ierarhie de clase cu moștenire publică (*is-a*)

Aplicația trebuie să conțină o **ierarhie de minimum 3 niveluri** folosind moștenire publică (`class Derivată : public Bază`), care să modeleze în mod logic domeniul ales.

> **Exemplu:** Dacă aplicația modelează o bibliotecă:
> `Material` → `Carte` → `CarteRara`
> sau
> `Persoană` → `Angajat` → `Manager`

Cerințe pentru ierarhie:

- Clasa de bază să conțină membri `protected` accesibili în clasele derivate.
- Fiecare clasă derivată să apeleze explicit constructorul bazei prin **lista de inițializare**.
- Ordinea de construcție și de distrucție să fie demonstrată prin mesaje în constructori/destructori.
- Cel puțin o clasă derivată să **suprascrie** o metodă a bazei, apelând versiunea bazei cu `Baza::metoda()` pentru a extinde (nu înlocui) comportamentul.

2. Compunere (*has-a*) cu inițializare prin listă

Cel puțin **una dintre clase** trebuie să conțină ca dată membră directă (nu pointer) un obiect al altei clase, demonstrând relația *has-a*.

- Componenta trebuie inițializată prin **lista de inițializare** a constructorului gazdei.
- Să se demonstreze că ordinea de construcție este: componenta înainte de corpul constructorului gazdei.

Exemplu: `Carte` are un obiect `Autor` ca dată membră directă;
`Autor` este inițializat în lista de inițializare a constructorului lui `Carte`.

3. Clasă abstractă și funcții pur virtuale

Aplicația trebuie să conțină **cel puțin o clasă abstractă** care:

- Declară **minimum 2 funcții pur virtuale** (`= 0`).
- Nu poate fi instanțiată direct (să fie demonstrat prin comentariu în cod).
- Poate conține metode concrete reutilizabile de derivate.
- Are **destructorul virtual** declarat.

Cel puțin **două clase concrete** să implementeze toate funcțiile pur virtuale, devenind instanțiabile.

4. Polimorfism prin pointeri/referințe la clasă de bază

- Să existe un tablou sau colecție de **pointeri la clasa de bază** (sau abstractă) care rețin obiecte de tipuri derivate diferite.
- Apelurile de metode virtuale prin acești pointeri să demonstreze **legatura tardivă** (late binding): fiecare obiect să execute propria implementare.
- Toate metodele suprascrise să fie marcate cu `override`.

5. Destructor virtual și gestionarea memoriei

- Orice clasă care are cel puțin o metodă `virtual` trebuie să declare **destructorul virtual**.
- Obiectele alocate dinamic prin `new` să fie eliberate corect prin `delete`, fără memory leaks.
- Să se demonstreze că la `delete` printr-un pointer la bază se apelează **ambii destructori** (al derivatei și al bazei), în ordinea corectă.

6. Downcasting cu `dynamic_cast`

Aplicația trebuie să conțină **cel puțin un exemplu** de downcast sigur folosind `dynamic_cast`:

- Să se verifice rezultatul (pointer non-null) înainte de utilizare.
- Să se trateze cazul eșecului (pointer null sau excepție `std::bad_cast`).
- Să fie justificat în comentarii de ce `dynamic_cast` este preferat față de `static_cast` în acel context.

7. Tipuri de moștenire și modificatori de acces

Proiectul trebuie să demonstreze înțelegerea modificatorilor de acces în moștenire:

- Cel puțin o clasă cu membri `private`, `protected` și `public`.

- Comentarii în cod care explică de ce un anumit acces este valid sau invalid.
- Opțional: folosirea moștenirii `protected` sau `private` acolo unde relația semantică o justifică.

8. Extinderea meniului interactiv

Meniul CRUD din Proiectul 1 trebuie extins pentru a acoperi și funcționalitățile noi:

- Crearea de obiecte din clase derivate diferite.
- Afișarea polimorfică a tuturor obiectelor dintr-o colecție.
- Operații specifice tipului concret (accesibile după `dynamic_cast`).

Cerințe suplimentare (bonus)

- Implementarea **moștenirii în diamant** cu `virtual public` și explicarea problemei rezolvate.
- Utilizarea `typeid` și `type_info` pentru inspecția tipului real al unui obiect polimorfic.
- Utilizarea corectă a `const_cast` pentru compatibilitate cu API-uri vechi.
- Utilizarea cuvintelor cheie `final` (pe clasă sau pe metodă) cu justificare în comentarii.
- Interfață grafică (ex: ImGUI) sau conectare la bază de date.

Condiții eliminatorii

NECOMPILAREA PROIECTULUI DUCE LA NOTAREA PROIECTULUI CU NOTA 1.

NERESPECTAREA PRINCIPIILOR DE ÎNCAPSULARE (atribute publice, lipsă getteri/setteri) DUCE LA NOTAREA PROIECTULUI CU NOTA 1.

ABSENȚA IERARHIEI DE MOȘTENIRE SAU A CLASEI ABSTRACTE DUCE LA NOTAREA PROIECTULUI CU NOTA 1.

Proiectul va fi verificat printr-un flux automatizat și prin **cross-evaluation** pe sursele predate.

Dacă în timpul rulării programul:

- Produce Undefined Behaviour (ex: `delete` prin pointer la bază fără destructor virtual, `static_cast` greșit, `const_cast` pe obiect `const`),
- Are funcționalități greșit implementate,
- Are funcționalități care eșuează la execuție,

se poate acorda punctaj parțial sau penalizare majoră.

Grilă de evaluare orientativă

Criteriu	Puncte
Ierarhie de moștenire publică (min. 3 niveluri, liste de inițializare)	15
Compunere cu inițializare prin listă (*has-a*)	10
Clasă abstractă cu funcții pur virtuale (min. 2)	15
Polimorfism prin pointeri la bază + `override`	15
Destructor virtual + gestionare corectă a memoriei	10
`dynamic_cast` cu tratarea eșecului	10
Modificatori de acces demonstrați cu comentarii	5
Menținerea cerințelor din Proiectul 1	10
Extinderea meniului CRUD pentru noile funcționalități	10
Total	**100**
Bonus (diamant, `typeid`, `final`, GUI, BD)	+10

Note tehnice

- Compilator: `g++ -std=c++17 -Wall -Wextra`
- Proiectul se predă ca arhivă `.zip` conținând toate fișierele sursă (`.cpp`, `.h`) și un fișier `README.txt` cu instrucțiuni de compilare și o scurtă descriere a aplicației.
- Se interzice utilizarea containerelor STL (`vector`, `list` etc.); gestionarea colecțiilor de obiecte se face cu tablouri sau pointeri alocați dinamic, ca în Proiectul 1.
- Fișierele sursă trebuie să fie organizate logic: câte un fișier `.h` și un fișier `.cpp` per clasă (sau grupate logic), plus un `main.cpp`.

Tabel sintetic al conceptelor acoperite

Concept	**Laborator sursă**	**Cerință asociată**
Compunere (*has-a*)	Lab 4, §1	Cerința 2
Moștenire publică (*is-a*)	Lab 4, §2–4	Cerința 1
Modificatori de acces	Lab 4, §3	Cerința 7
Constructori în ierarhii	Lab 4, §5	Cerințele 1, 2
Funcții virtuale + `override`	Lab 5, §1	Cerința 4
Destructor virtual	Lab 5, §1.4	Cerința 5
Clase abstracte / pur virtuale	Lab 5, §2	Cerința 3
Upcasting / Downcasting	Lab 5, §3	Cerința 6
`dynamic_cast`	Lab 5, §3.2.3	Cerința 6
Moștenire în diamant	Lab 5, §5	Bonus